COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Start the native-engineering bundle

Continuation guide and build workflow

Where Package 04 fits, what to run, and what constitutes student evidence.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

Where this bundle belongs 3

Readiness before Lesson 11.1 3

Open these files 4

Toolchain: use the computer already available 4

First reference run 5

Student work: the initial failures are intentional 5

Study and assessment loop 6

Release, memory tools and benchmark separation 6

Preserve evidence and boundaries 7

Start Package 04

Where this bundle belongs

Package 04 expands Volume 03: C++, Algorithms and Language Implementation. It follows Package 03 / Volume 02 Mathematics and preserves Modules 11–15, lessons 11.1–15.4 and Gate C. Package numbering is not volume numbering. It does not replace or edit the separately authored Package 03.

This is the undergraduate computer-science core of the larger undergraduate-to-graduate game-engineering route. Here you learn native execution, ownership, data structures, algorithms and language implementation. Advanced operating systems, graphics, digital design and graduate research remain in later volumes. The bundle is a self-study course, not an accredited degree or a claim of completed university laboratory credit.

The original Volume 03 roadmap assigns **400–650 planning hours**, including study, exercises, projects, review and Gate C. That is not the time needed to read the PDFs. The following allocation is a planning aid, not a prescribed schedule. The five module projects feed one Gate C dossier; do not add a sixth full project allocation.

Module	Workstream	Planning hours
11	Native programming and grid port	60–90
12	Ownership and resource manager	70–110
13	Containers and cost analysis	80–130
14	Algorithms and pathfinding tools	90–150
15	Language runtime, C bridge and Gate C integration	100–170

Readiness before Lesson 11.1

The ordinary route completes the earlier volume checkpoints. Module 11 specifically requires Volume 01 programming skills and Modules 06–07 mathematical foundations. You should be able to run a saved program, write a function and a loop, trace a list mutation, test a boundary, use Git and explain an invariant. The later algorithm lessons require discrete proofs and graph reasoning, not only familiarity with game-editor tools.

Package production can happen in parallel; learner prerequisites still apply. While Package 03 is being written or studied, it is reasonable to inspect the native toolchain and review the beginning of this volume. Do not treat that overlap as permission to skip the mathematical prerequisites of Modules 13–15.

A prior-learning route requires the same independent evidence: explain a changed example, demonstrate the project behavior and identify your own weak prerequisites. Recognition of syntax is not a placement test. Record a missing skill and one small repair exercise in the prerequisite log.

Open these files

Extract the entire ZIP, keeping its folder structure. Begin with this guide, then open **STUDENT_HANDBOOK.pdf** or **COURSE.html**. The browser edition works without a server or internet connection after extraction. External reading links still need internet access; the cited textbooks and university courses are not copied into this ZIP.

File	Purpose
STUDENT_HANDBOOK.pdf	20 expanded lessons, worked examples, labs and five-question quizzes
LAB_WORKBOOK.pdf	Full lab contracts, 40 extra exercises, 25 reviews and five module projects
ANSWER_KEY.pdf	Separate original/application answers, exercise solutions, reviews and C-bridge answers
CAPSTONE_SPEC.pdf	Gate C milestones, acceptance tests, rubric and oral review
C_BRIDGE.pdf	Four actual-C/C++ boundary exercises for the hardware route
CPP_REFERENCE.pdf	Types, ownership, containers, complexity, grammar, limits and troubleshooting
modules/	Five separately readable module PDFs
code/reference/	Complete runnable teaching demonstrations, not student submissions
code/starter/	20 intentionally unfinished student contracts
source/	Editable Markdown, lesson JSON and original Volume 03 manifest extract
templates/	Journals, ownership, benchmark, review and handoff templates
validation/	Actual test transcripts, environment and validation scope

Toolchain: use the computer already available

You need a C++20-capable C++ compiler, a C17-capable C compiler, their standard libraries, **CMake 3.20 or later**, and a supported CMake build backend. No GPU, engine SDK, cloud account, LLVM development libraries, FPGA or new hardware is needed. The LLVM reading explains compiler architecture; the included expression compiler is standalone C++ and does not link LLVM. Optional helper scripts use the Python standard library, with Python 3.11+ as the selected baseline. Direct CMake commands do not require Python. [R11; R12; R14; P4-01; P4-05]

Check `cmake --version` first. CMake's configure output identifies the chosen C and C++ compilers. On a system with multiple toolchains, select one deliberately and record it. A Windows native toolchain may require its configured developer terminal. A missing compiler is an environment problem, not a reason to remove the language standard or warning requirements. Use your platform's official compiler installation instructions and documentation matching its release.

The package was exercised on Linux with GCC 14.2.0, Clang 17.0.0 and CMake 3.31.6. Debug, Release and a Clang AddressSanitizer/UndefinedBehaviorSanitizer configuration passed the reference suite. Windows, macOS, other compiler versions and hardware-device integration were not separately executed here. See VALIDATION.md for the exact scope and logs.

First reference run

Open a terminal in the extracted **Game_Engineering_Package_04** folder. These commands support the usual single- and multi-configuration CMake workflows; a single-configuration generator uses CMAKE_BUILD_TYPE, while a multi-configuration generator uses `--config` and `-C`.

```
cmake -S code -B build/debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug --config Debug --parallel 2
ctest --test-dir build/debug -C Debug -L reference --output-on-failure
```

The expected default result is **24 reference test programs passing**: 20 lesson programs and four C-bridge programs. Each program contains multiple checks. Those are not 24 completed student assignments, a security certification or a proof that every input has been tested.

Run one lesson through CTest without finding the platform-specific executable suffix or configuration subdirectory:

```
ctest --test-dir build/debug -C Debug -R "^reference_11_1$" -V
```

An optional wrapper performs the same reference configure/build/test sequence:

```
python3 code/tools/run_reference.py --build build/debug --config Debug
```

Use the working Python command on your system, such as `python` or `py -3`, when `python3` is not available. Do not paste the shell prompt characters into the command. Python is not the C++ compiler.

Student work: the initial failures are intentional

After studying and predicting the example, open `code/starter/lesson_11_1.cpp`. Each starter supplies a narrow initial contract and tests. **It compiles but deliberately fails with a TODO message until you implement it.** It does not contain every required test, every helper function or a finished module project. Extend it using the workbook, not just the visible assertions.

```
cmake -S code -B build/student -DCMAKE_BUILD_TYPE=Debug -DP4_BUILD_STUDENT=ON
cmake --build build/student --config Debug --target student_11_1
ctest --test-dir build/student -C Debug -R "^student_11_1$" --output-on-failure
```

Repeat for the lesson being studied. Building all student targets initially succeeds, but running all 20 student tests initially fails. Do not remove their failing tests, call a supplied reference implementation as the answer, or change expected values merely to get a green status. A small initial starter is a seed for the full lab, not a replacement for it.

For Lesson 11.1, split your implementation into a declaration header and a separate `.cpp`, then add the extra source to the student target with `target_sources`. To see controlled compile/link failures with a GCC/Clang-style driver:

```
python3 code/tools/diagnose_build.py --cxx c++
```

That diagnostic helper verifies a good baseline, a syntax rejection and an unresolved-symbol rejection. It never runs the deliberately broken sources. Its flags are not a claim of support for the MSVC command-line syntax.

Study and assessment loop

For each lesson, recall the prerequisite, predict the worked result, study the explanation and assigned primary reading, implement the lab, test the required cases, answer the five questions, then consult the separate key. Save code, commands, results and reasoning. Revisit a missed concept using the independent transfer prompt rather than memorizing its answer.

Questions 1–3 are the **60 original questions**, retained with their original answers. Score each 0–2: incorrect/absent, partially correct, or correct with reasoning. Preserve the original formative threshold of **5/6**. Questions 4–5 are **40 new application questions**. A suggested expansion threshold is **8/10 overall**, while still meeting 5/6 on the original core. These are formative checks, not validated exams.

A lesson also needs its practical pass condition and assigned study. Every module adds **eight original extra exercises** and **five module-review questions**. Create **five new edge/failure cases of your own per module**, beyond the supplied examples, and explain what each could expose. Those learner-designed cases are requirements, not pre-authored tests being counted twice.

Use the 20 independent transfer prompts for review without following a walkthrough. Module projects culminate in Gate C. The four C-bridge exercises add 12 short questions, counted separately from the 100 lesson questions and 25 module reviews. Keep the answer key closed until an independent attempt.

Release, memory tools and benchmark separation

```
cmake -S code -B build/release -DCMAKE_BUILD_TYPE=Release
cmake --build build/release --config Release --parallel 2
ctest --test-dir build/release -C Release -L reference --output-on-failure
```

On a GCC/Clang toolchain with the documented sanitizer runtime, use a new build directory:

```
cmake -S code -B build/sanitize -DCMAKE_BUILD_TYPE=Debug -DP4_SANITIZE=ON
cmake --build build/sanitize --config Debug --parallel 2
ctest --test-dir build/sanitize -C Debug -L reference --output-on-failure
```

Sanitizers test executed paths and add overhead; they do not prove all lifetimes safe or replace invariants. Do not compare sanitized Debug timings to uninstrumented Release timings as an algorithm comparison. The test harness's CHECK remains active in Release. [P4-03; P4-04]

The `layout_benchmark` executable emits CSV. With a single-configuration generator, a typical invocation is `build/release/layout_benchmark 100000 100`; with a multi-configuration generator, find it in the Release subdirectory and use the platform's executable suffix as needed. Its numbers describe a narrow x/vx update workload, not a recommendation that every engine must use one layout.

Preserve evidence and boundaries

Use `templates/LAB_JOURNAL.md` and commit meaningful steps. Document generated or borrowed help and prove that you can explain the central implementation. Do not include credentials, employer code or proprietary SDK material. Teaching mock devices perform no physical I/O. The tiny VM has no host callbacks, memory pointers or filesystem access and is not a production security sandbox.

At the end of this volume, your deliverable is your own tested native simulation and bounded language system with an engineering explanation. The supplied code provides examples and comparison evidence; it does not pre-complete that work for you.